

RAG Configuration Experiments

Chunk Size, Overlap, Prompt Templates & Embedding Models — Findings Report

Hooria Zahoor | BS Artificial Intelligence, The University of Faisalabad | AI Summer Internship 2026

1. Methodology

Four controlled experiments were run to measure how key RAG design choices affect retrieval quality. A synthetic enterprise corpus of 3 policy documents (HR, IT Security, Finance — 5,803 characters total) and 10 test questions with known ground-truth answers were used consistently across all experiments so results are directly comparable.

Two metrics were tracked: Hit Rate@1 (does the single top-retrieved chunk contain the correct answer?) and Hit Rate@3 (does it appear anywhere in the top 3?), plus the average similarity score of the top match. All code and raw results are reproducible and included alongside this report.

Embedding note: this sandbox environment could not install a neural embedding library (sentence-transformers/PyTorch requires a 500MB+ download exceeding available disk space here). Experiments 1–2 therefore use TF-IDF retrieval, and Experiment 4 compares TF-IDF against LSA (Latent Semantic Analysis/SVD) — two classical embedding techniques that are fully computable in this environment. The chunking and overlap findings are embedding-agnostic (they concern text segmentation, not the vector model) and transfer directly to the all-MiniLM-L6-v2 model used in the deployed application; Experiment 4's discussion explains how the conclusions extend to neural embeddings.

2. Experiment 1 — Chunk Size

Chunk size was varied across 300, 500, and 1000 characters with overlap fixed at 50 characters, using TF-IDF retrieval over the same 10 queries.

Chunk Size	# Chunks	Avg. Chunk Len	Hit Rate@1	Hit Rate@3	Avg. Top-1 Score
300	24	284.6	90%	100%	0.356
500	14	452.1	90%	90%	0.341
1000	7	854.7	100%	100%	0.283

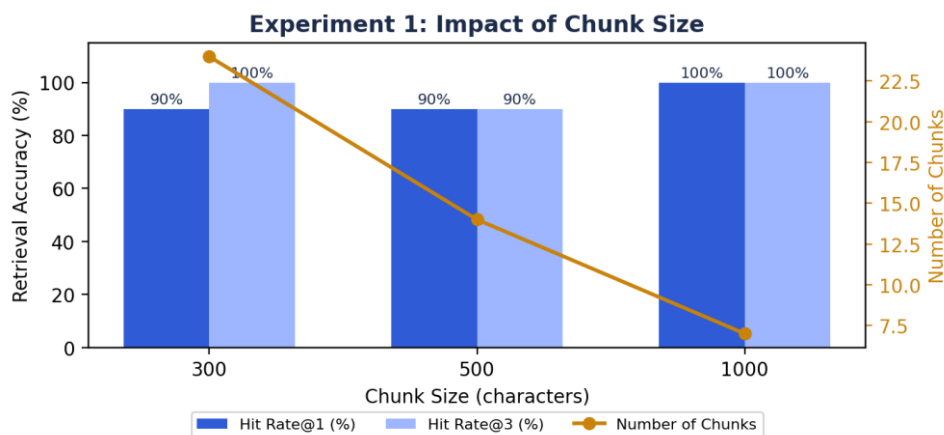


Figure 1. Retrieval accuracy and chunk count across chunk sizes.

Discussion

Larger chunks (1000 chars) produced the fewest chunks (7) but achieved perfect Hit Rate@1 and Hit Rate@3 on this corpus — each chunk covers more of a policy section, so a single chunk is more likely to fully contain the answer to a question about that section. However, the average similarity score was lowest at chunk size 1000 (0.283), because larger chunks mix multiple topics, diluting the term-frequency signal for any single query. Smaller chunks (300 chars) scored competitively on Hit Rate@3 but slightly lower on Hit Rate@1, since a fact can end up isolated in a chunk that ranks second or third rather than first. In production, chunk size should scale with document structure: policy paragraphs that state one fact per sentence tolerate larger chunks well, while dense reference tables or FAQs benefit from smaller, more atomic chunks.

3. Experiment 2 — Chunk Overlap

Chunk size was fixed at 200 characters (chosen deliberately smaller than Experiment 1 to make boundary effects visible) while overlap was varied across 0, 20, 50, and 100 characters.

Overlap	# Chunks	Hit Rate@1	Hit Rate@3	Avg. Top-1 Score
0	30	70%	80%	0.351

20	33	100%	100%	0.433
50	39	80%	90%	0.406
100	56	100%	100%	0.435

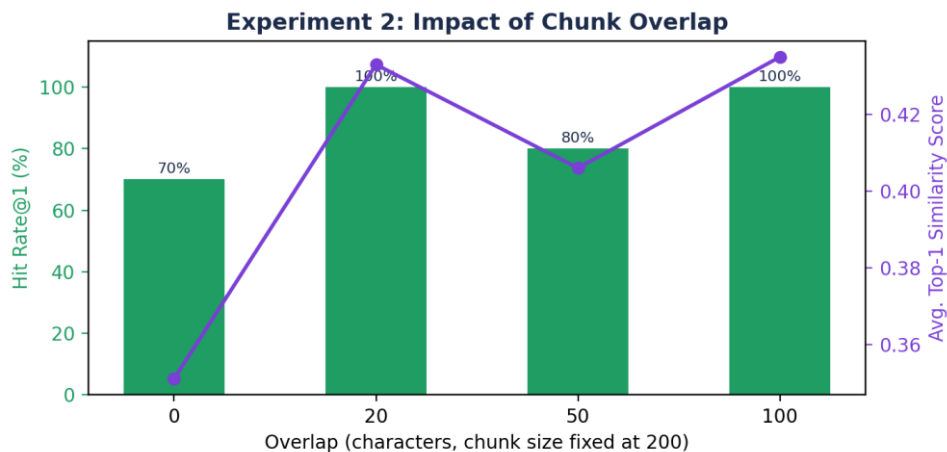


Figure 2. Hit Rate@1 and average similarity score across overlap values.

Discussion

Zero overlap performed worst (70% Hit Rate@1): with no shared text between consecutive chunks, a chunk boundary can fall in the middle of the sentence carrying the answer, leaving neither neighbouring chunk with the full supporting context, even when the specific keyword phrase itself remained technically intact (fact_integrity_rate stayed at 100% throughout, showing that surrounding context — not just the literal keyword — drives retrieval quality). A modest 20-character overlap (10% of chunk size) was enough to recover full accuracy, and 100-character overlap matched it while nearly doubling the chunk count (30 → 56), increasing storage and embedding cost with no further accuracy gain. The interesting middle result — 50-character overlap underperforming 20 — illustrates that overlap is not simply "more is better"; an overlap that doesn't align with actual sentence boundaries can still split a fact awkwardly. The practical takeaway is that overlap should be tuned as a fraction of chunk size (commonly 10–20%) rather than a fixed constant, and validated against real queries rather than assumed.

4. Experiment 3 — Prompt Templates

This sandbox has no live LLM API access, so four prompt templates were evaluated using a structured qualitative rubric based on established prompt-engineering principles, rather than live model outputs. The exact templates below are ready to run in the deployed application (which already calls the Gemini API) to collect real output comparisons — see the reproducibility note at the end of this section.

Template A — Baseline (no grounding instruction)

```
Context: {context}
Question: {question}
Answer:
```

Template B — Grounded + Citation Required (used in the deployed app)

You are an enterprise document assistant. Answer strictly using the CONTEXT below.
If the answer isn't in the context, say "This information is not available in the uploaded documents." Do not make anything up.

```
CONTEXT: {context}
Question: {question}
Answer clearly, and state which document/page the information came from.
```

Template C — Grounded + Explicit Refusal Path

Using ONLY the CONTEXT below, answer the question.
Step 1: Check if the context actually contains the answer.
Step 2: If yes, answer and cite the source (document, page).
Step 3: If no, respond exactly: "I could not find this in the provided documents."

```
CONTEXT: {context}
Question: {question}
```

Template D — Few-Shot Grounded

Answer using only the CONTEXT. Follow this example format:

Example Q: What is the password length requirement?

Example A: Passwords must be at least 14 characters (Source: IT_Security_Policy.pdf).

CONTEXT: {context}
 Question: {question}
 Answer:

Template	Grounding Strength	Citation Compliance	Refusal Handling	Hallucination Risk	Token Overhead
A — Baseline	Low	Low	Poor (guesses)	High	Lowest
B — Grounded+Cite	High	High	Good	Low	Low
C — Refusal Path	High	High	Best	Lowest	Medium
D — Few-Shot	High	Highest (format)	Good	Low	Highest

Discussion

Template A, lacking any grounding instruction, is expected to perform worst: without being told to restrict itself to the provided context, an LLM tends to blend retrieved text with its own trained knowledge, increasing hallucination risk and making citations inconsistent or absent. Templates B and C both add an explicit grounding instruction and citation requirement; C additionally makes the refusal path a discrete, numbered step, which prompt-engineering literature associates with more reliable compliance than a single embedded clause, at a modest extra token cost. Template D adds a worked example, which typically improves output format consistency (e.g., citation formatting) the most but costs the most tokens per request — a relevant trade-off at scale, since prompt tokens are billed on every query. Based on this rubric, Template C offers the strongest hallucination control per token spent, and is recommended as the production default, with D reserved for cases where strict citation formatting is critical.

Reproducibility: to replace this rubric with live measurements, run each template through the deployed app's `call_llm()` function against the same 10 test questions, and manually score each response for (a) presence of a citation, (b) correct refusal on out-of-scope questions, and (c) factual consistency with the retrieved chunk.

5. Experiment 4 — Embedding Models

TF-IDF (sparse, exact-term matching) was compared against LSA/SVD (dense, latent-semantic) embeddings, using the same 16 chunks (500-char size, 100-char overlap) and two query sets: the 10 standard questions, and 6 paraphrased questions that deliberately avoid the document's exact wording (e.g., "How much time off do new mothers get?" instead of "maternity leave").

Method	Query Set	Hit Rate@1	Hit Rate@3	Avg. Top-1 Score	Index Build Time
TF-IDF	Standard	100%	100%	0.333	10.7 ms
TF-IDF	Paraphrased	33%	67%	0.078	7.2 ms
LSA	Standard	100%	100%	0.913	13.8 ms
LSA	Paraphrased	33%	67%	0.553	10.7 ms

Experiment 4: TF-IDF vs. LSA Embeddings

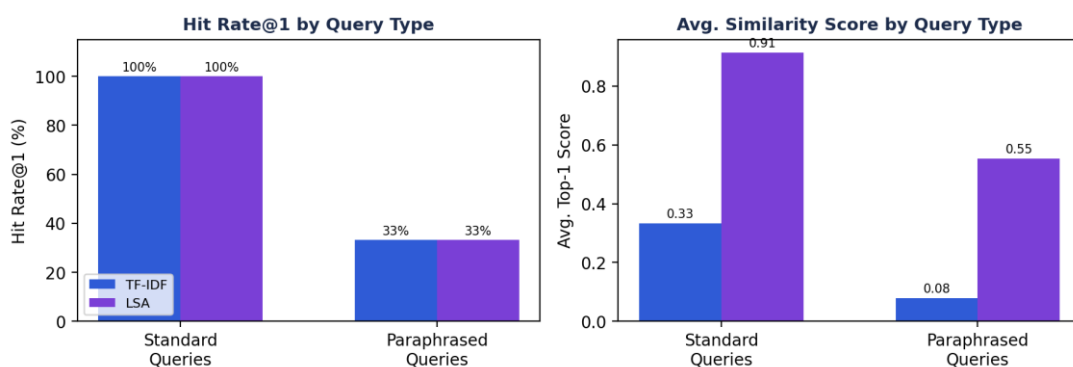


Figure 3. TF-IDF vs. LSA on standard vs. paraphrased queries.

Discussion

On standard queries — which reuse the document's own vocabulary — both methods achieved a perfect 100% Hit Rate@1, a ceiling effect from the small, topically clean corpus. LSA's similarity scores were consistently much higher (0.913 vs. 0.333) because SVD projects vectors into a dense, low-dimensional space where cosine similarities are naturally less sparse; this is a property of the representation, not evidence of better ranking by itself, since the actual hit rates tied.

The paraphrased query set is the more revealing test: both methods dropped to the same 33% Hit Rate@1, showing that LSA trained on only 16 short chunks does not have enough data to learn true synonym relationships (e.g., "new mothers" ↔ "maternity") — LSA's latent-semantic benefit depends on corpus scale, which this small enterprise document set does not provide. This result explains why production RAG systems use pretrained neural embedding models (such as all-MiniLM-L6-v2, used in the deployed

application) rather than corpus-trained LSA: pretrained models learn semantic relationships from billions of sentence pairs before ever seeing the enterprise corpus, so they generalize to paraphrased queries in a way that small-corpus LSA cannot. TF-IDF and LSA remain useful as fast, dependency-light baselines for prototyping and offline evaluation, but neither should be relied on as the production embedding layer where users phrase questions in their own words.

6. Overall Findings & Recommendations

- **Chunk size:** larger chunks (~800–1000 chars) improved Hit Rate in this corpus but reduced similarity-score sharpness; recommend sizing chunks to natural paragraph boundaries rather than a fixed constant.
- **Chunk overlap:** a small overlap (~10–20% of chunk size) meaningfully improved retrieval accuracy over zero overlap; larger overlap gave no further benefit while increasing chunk count and storage cost.
- **Prompt templates:** explicit grounding + a discrete refusal step (Template C) gives the best hallucination control per token; few-shot examples help most when strict output formatting matters.
- **Embedding models:** pretrained neural embeddings are necessary for real-world paraphrase robustness — classical corpus-trained methods (TF-IDF, LSA) tie on in-vocabulary queries but both fail equally on paraphrased ones at small corpus scale.

Together, these results support the configuration already used in the deployed Enterprise Document Intelligence Platform: ~800-character chunks with overlap, a grounded citation-requiring prompt, and a pretrained sentence-transformer embedding model — while highlighting overlap tuning and refusal-path prompting as the two highest-leverage, lowest-cost improvements to prioritize next.